

# CVE-2025-14187\_漏洞原理分析

## CVE-2025-14187 漏洞原理分析报告

### UGREEN DH2100+ NAS nas\_svr 组件缓冲区溢出

| 文档版本 | 日期         | 作者           |
|------|------------|--------------|
| v1.0 | 2026-06-04 | Hermes Agent |

## 1. 漏洞概述

### 1.1 基本信息

| 字段       | 内容                                               |
|----------|--------------------------------------------------|
| CVE ID   | CVE-2025-14187                                   |
| 漏洞类型     | 栈缓冲区溢出 (Stack-based Buffer Overflow)             |
| CWE 分类   | CWE-119: 内存缓冲区操作限制不当                             |
|          | CWE-120: 未检查输入大小的缓冲区复制 (Classic Buffer Overflow) |
| CVSS 3.1 | 7.2 (HIGH) — AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H |
| 受影响设备    | 绿联 UGREEN DH2100+ 私有云 NAS                        |
| 受影响固件    | ≤ 5.3.0.251125                                   |
| 修复版本     | 5.3.0.251125 以上                                  |
| 公开日期     | 2025 年 7 月                                       |
| 发现/提交者   | VulDB                                            |

### 1.2 影响评估

|        |                        |
|--------|------------------------|
| 攻击复杂度： | 低（AC:L）                |
| 攻击路径：  | 远程网络（AV:N）             |
| 所需权限：  | 高（PR:H）－ 需要有效 NAS 登录凭证 |
| 影响范围：  | 不变（S:U）                |
| 机密性：   | 完全丧失（C:H）              |

完整性： 完全丧失 (I:H)  
可用性： 完全丧失 (A:H)

## 2. 受影响的组件架构

### 2.1 设备概况

UGREEN DH2100+ 是一款双盘位 NAS 设备，核心架构如下：

SoC: Realtek RTD1296  
CPU: ARM Cortex-A53 四核 64-bit (aarch64)  
内存: 2GB DDR4  
存储: 双 3.5" SATA 盘位  
系统: 定制 Linux (内核版本待定)

### 2.2 关键软件组件

设备运行多个后台服务，其中 **nas\_svr** 是核心管理守护进程：

```
nas_svr (核心管理服务)
├── HTTP/HTTPS API 服务 (监听 80/443)
│   ├── /v1/file/backup/create    ← 漏洞端点
│   ├── /v1/file/backup/list
│   ├── /v1/file/backup/delete
│   └── /v1/file/...              (其他文件管理 API)
├── 认证模块 (Basic Auth / Token)
├── 文件系统管理
├── RAID/存储管理
└── 任务调度 (备份任务等)
```

## 3. 漏洞原理详解

### 3.1 攻击入口

漏洞位于 `nas_svr` 组件对外暴露的 HTTP API 端点：

```
POST /v1/file/backup/create
Content-Type: application/x-www-form-urlencoded
Authorization: Basic <credentials>
```

path=<用户指定的备份路径>

该端点提供备份功能，用户指定要备份的目录/文件路径，服务端创建备份任务。

## 3.2 触发路径

攻击者

1. 登录 NAS (获取认证凭证)
2. POST /v1/file/backup/create
  - Header: Authorization
  - Body: path = <恶意超长字符串>
3. nas\_svr 收到请求
  - router 路由到 handler\_file\_backup\_create()
4. 函数解析 path 参数
  - 从 POST body 中提取 path 字符串
5. 将 path 复制到栈缓冲区
  - strcpy() / sprintf() / memcpy() ← 未检查长度
6. 缓冲区溢出
  - 覆盖相邻栈变量
  - 覆盖 saved FP (帧指针)
  - 覆盖 return address (返回地址)
7. 函数返回时
  - PC 跳转到攻击者控制地址 → RCE

## 3.3 漏洞代码模式 (推测)

根据 CWE-120 典型模式，漏洞代码可能形如：

```
void handler_file_backup_create(const Request *req) {
    char path_buf[256];          // 固定大小栈缓冲区
    const char *user_path;

    user_path = get_post_param(req, "path");
    // 错误：没有检查 user_path 长度！
    strcpy(path_buf, user_path); // ← 缓冲区溢出发生点
    //      ^^^^^^^^^^^^^^^^^^^
    // 如果 user_path > 255 字节，溢出开始
```

```

// ... 后续处理 path_buf ...
create_backup_task(path_buf);
//      ^                               返回时，覆盖的返回地址被使用
//      └ 函数 epilogue:
//          ldp x29, x30, [sp], #N
//          ret                                ← PC 跳转到攻击者地址
}

```

**安全的写法应该是：**

```

void handler_file_backup_create(const Request *req) {
    char path_buf[256];
    const char *user_path;

    user_path = get_post_param(req, "path");
    if (strlen(user_path) >= sizeof(path_buf)) {
        // 正确做法：拒绝过长的输入
        send_error(req, 400, "Path too long");
        return;
    }
    strcpy(path_buf, user_path); // 安全，已检查长度
    // 或使用 strncpy(path_buf, user_path, sizeof(path_buf)-1);
    create_backup_task(path_buf);
}

```

## 3.4 栈布局变化

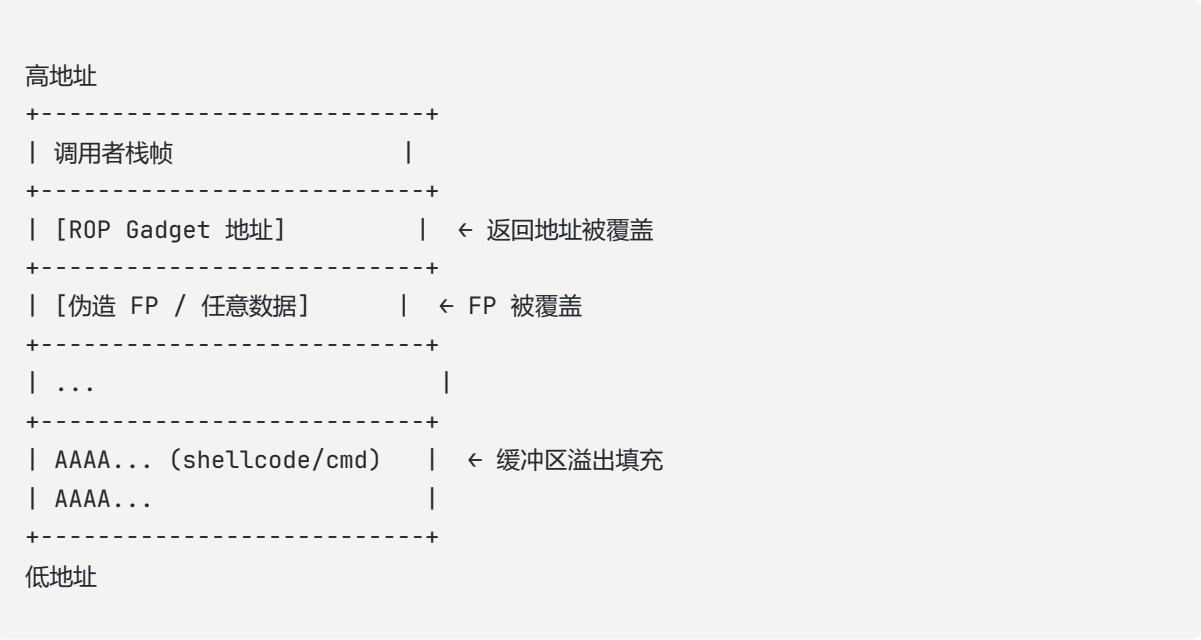
**正常执行时的栈布局：**

```

高地址
+-----+
| 调用者栈帧          |
+-----+
| 函数返回地址 (RET)   | ← x30 (LR)
+-----+
| 保存的帧指针 (FP)   | ← x29
+-----+
| 局部变量 ...        |
+-----+
| path_buf[255]       | ← 栈顶向下增长
| path_buf[0]         | ← SP
+-----+
低地址

```

攻击后的栈布局：



## 4. ARM64 函数调用与返回流程

要理解这个漏洞如何从溢出变为 RCE，需要了解 ARM64 (AArch64) 的函数调用约定。

### 4.1 关键寄存器

| 寄存器      | 用途                             |
|----------|--------------------------------|
| x0-x7    | 参数传递 (x0 = 第一个参数)              |
| x29 (FP) | 帧指针 (Frame Pointer)            |
| x30 (LR) | 链接寄存器 (Link Register) — 保存返回地址 |
| SP       | 栈指针                            |
| PC       | 程序计数器                          |

### 4.2 函数序言 (Prologue)

```
handler_file_backup_create:
    stp    x29, x30, [sp, #-N]!    ; 保存 FP 和 LR 到栈上, 分配 N 字节
    mov    x29, sp                ; 设置新的帧指针
    sub    sp, sp, #0x100         ; 再分配 256 字节局部变量空间
```

### 4.3 函数结语 (Epilogue)

```

add    sp, sp, #0x100          ; 回收局部变量空间
ldp    x29, x30, [sp], #N      ; 恢复 FP 和 LR, 回收栈帧
ret                                ; 跳转到 x30 (LR)

```

**漏洞本质:** 当 `path` 参数长度超过 `path_buf` 大小时, `strcpy()` 持续写入, 依次覆盖:

1. `path_buf[256..511]` — 覆盖其他局部变量
2. 从 `sp + N` 开始 — 覆盖保存的 `x29` (FP)
3. 从 `sp + N + 8` 开始 — 覆盖保存的 `x30` (LR/返回地址)
4. 再往上 — 覆盖调用者的栈帧

当函数执行 `ldp x29, x30, [sp], #N` 时, `x30` 已被攻击者控制值替代。

随后 `ret` 指令将 PC 设置为攻击者指定的地址 → **代码执行**。

## 5. 利用路径分析

### 5.1 前置条件

| 条件       | 说明                         | 可行性           |
|----------|----------------------------|---------------|
| 网络可达     | 攻击者能访问 NAS 的 HTTP/HTTPS 端口 | 公网暴露的 NAS 满足  |
| 有效凭证     | PR:H 要求高权限用户               | 默认密码未改的设备满足   |
| 无 canary | 或能 leak canary             | 待逆向确认         |
| 无 ASLR   | 或能 leak 基址                 | 嵌入式设备常关闭 ASLR |

### 5.2 利用策略

策略 1: `ret2libc` (推荐)

[填充] [`x30 = system@plt`] [`x0 = "/bin/sh" 指针`]  
 └ 最稳定, 不需要 shellcode

策略 2: ROP Chain

[填充] [`gadget1`] [`gadget2`] ... [最后跳转到 shellcode]  
 └ 需要足够多 gadgets, 但灵活性最高

策略 3: `stack pivot` (当缓冲区不够大时)

[填充] [`leave; ret gadget`] [伪造栈帧]  
 └ 将 SP 移到攻击者控制的更大内存区域

## 5.3 攻击成功后的影响

一旦 RCE 成功，攻击者可：

- 完全控制 NAS 文件系统（读取/修改/删除所有数据）
- 植入后门/持久化（cron, SSH key, 修改 nas\_svr）
- 横向移动（NAS 在内网，作为跳板）
- 加密勒索（加密所有存储数据）
- 加入僵尸网络（DDoS 等）
- 窃取敏感数据（备份文件通常包含完整用户数据）

## 6. CWE 技术细节

### 6.1 CWE-119 vs CWE-120

| 特性  | CWE-119       | CWE-120       |
|-----|---------------|---------------|
| 全称  | 内存缓冲区操作限制不当   | 未检查输入大小的缓冲区复制 |
| 范围  | 更广泛, 包括各类内存操作 | 特指复制操作中的长度缺失  |
| 本漏洞 | ✓ (父类)        | ✓ (子类, 精确分类)  |

### 6.2 典型易受攻击函数

| 危险函数                                  | 安全替代                                                       |
|---------------------------------------|------------------------------------------------------------|
| <code>strcpy(dst, src)</code>         | <code>strncpy(dst, src, n)</code> / <code>strlcpy()</code> |
| <code>sprintf(dst, "%s", src)</code>  | <code>snprintf(dst, n, "%s", src)</code>                   |
| <code>memcpy(dst, src, n_user)</code> | 需验证 <code>n_user ≤ dst_size</code>                         |
| <code>gets(buf)</code>                | <code>fgets(buf, n, stdin)</code>                          |
| <code>scanf("%s", buf)</code>         | <code>scanf("%Ns", buf)</code>                             |

## 7. 修复建议

### 7.1 供应商 (绿联)

1. 代码层面: 在 `handler_file_backup_create` 中增加 `path` 参数长度校验
2. 编译层面: 开启 Stack Canary ( `-fstack-protector-strong` )

3. **系统层面**: 启用 ASLR, NX (不可执行栈)
4. **加固**: 最小权限原则, nas\_svr 不应以 root 运行

## 7.2 用户层面

1. **升级固件**至 5.3.0.251125 以上版本
2. **更改默认密码** (admin/admin 等)
3. **不在公网暴露 NAS 管理端口** (使用 VPN 替代)
4. **防火墙策略**: 限制对 443/80 端口的来源 IP
5. **定期备份**: 即使被攻破也能恢复数据

## 8. 时间线

| 时间         | 事件                         |
|------------|----------------------------|
| 2025-07    | VulDB 发现并记录漏洞 (ID: 334607) |
| 2025-07-12 | NVD 收录 CVE-2025-14187      |
| 2025-07    | Exploit 提交到 VulDB 并公开      |
| 2025-07    | 绿联发布修复固件 5.3.0.251125      |
| 2026-04-14 | NVD 最后更新                   |

## 附录 A: 术语表

| 术语                          | 说明                                          |
|-----------------------------|---------------------------------------------|
| <b>Buffer Overflow</b>      | 缓冲区溢出 — 写入数据超过缓冲区边界                         |
| <b>Stack</b>                | 栈 — 存储局部变量和函数调用信息的 LIFO 内存区域                |
| <b>RET / Return Address</b> | 返回地址 — 函数结束后恢复执行的地址                         |
| <b>ROP</b>                  | Return-Oriented Programming — 利用现有指令片段构建攻击链 |
| <b>Canary</b>               | 栈保护值 — 检测溢出的编译时保护机制                         |
| <b>ASLR</b>                 | 地址空间布局随机化 — 随机化内存基址                         |
| <b>NX / DEP</b>             | 不可执行栈 / 数据执行保护 — 阻止栈上执行代码                   |
| <b>Gadget</b>               | 以 ret 结尾的指令片段, 用于 ROP 链                     |

## 附录 B: 参考链接

- NVD: <https://nvd.nist.gov/vuln/detail/CVE-2025-14187>
- VulDB (need auth): <https://vuldb.com/?id.334607>
- CWE-119: <https://cwe.mitre.org/data/definitions/119.html>
- CWE-120: <https://cwe.mitre.org/data/definitions/120.html>
- UGREEN: <https://www.ugreen.com>